Java Web & REST API Development:

1. Java Web Development

Definition: Java Web Development is the process of creating websites and web applications using Java.

Example: Creating a student management website to store and display student details.

2. REST API

Definition: A REST API allows two applications to communicate with each other over the internet using HTTP methods.

Example: A weather application uses an API to get current weather information from a server.

Learn HTTP routing protocols and Spring Boot REST API web services framework:

1. HTTP Routing Protocols

Definition: HTTP routing is the process of directing client requests to the correct URL or endpoint in a web application using HTTP methods such as GET, POST, PUT, and DELETE.

Example:

```
@GetMapping("/students")
public String getStudents() {
    return "Student Details";
}
```

Output:

Student Details

Explanation: When a user sends a GET request to /students, the application returns the student details message.

2. Spring Boot REST API Web Services Framework

Definition: Spring Boot is a Java framework used to build REST APIs and web services easily, allowing applications to exchange data over HTTP.

Example:

```
import org.springframework.web.bind.annotation.GetMapping;

import org.springframework.web.bind.annotation.RestController;

@RestController

public class HelloController {

    @GetMapping("/hello")

    public String sayHello() {

        return "Welcome to Spring Boot!";

    }

}
```

Output:

Welcome to Spring Boot!

Explanation: When a user sends a GET request to /hello, Spring Boot returns the message.

In simple words:

- HTTP Routing: Directs requests to the correct endpoint.
- Spring Boot REST API: Helps create web services that allow applications to communicate with each other.

Implement MVC architecture: Controllers, Services, Repositories, and JSON processing:

MVC Architecture in Java Spring Boot

1. Simple Definition

MVC (Model-View-Controller) is a software design pattern that separates an application into different parts to make the code easier to understand, develop, and maintain.

2. Main Components

- Controller: Receives user requests and returns responses.
- Service: Contains the main business logic.
- Repository: Communicates with the database to store and retrieve data.
- JSON Processing: Converts data into JSON format so applications can exchange information easily.

3. Simple Example: Student Management System

Suppose we want to display a student's name using a Spring Boot REST API.

Controller.java

```
@RestController

public class StudentController {

   @GetMapping("/student")

   public String getStudent() {

      return "Srilatha";

   }
```

}

Output:

Srilatha

Here, the controller receives the request and returns the student's name.

4. How the Components Work Together

Client Request

Requests student details

Controller

Handles the request

Service

Processes the business logic

Repository

Fetches or saves database records

JSON Response

Returns data to the client

5. Example of JSON Processing

JSON (JavaScript Object Notation) is a format used to exchange data.

```
{
  "id": 1,
  "name": "Srilatha",
```

```
  "course": "B.Tech CSE"
}
```

This JSON object contains a student's ID, name, and course.

In simple words: The Controller handles requests, the Service processes logic, the Repository manages database operations, and JSON formats the data for communication.

Establish JPA/Hibernate database connections, validation, and custom error handling blocks:

1. JPA/Hibernate Database Connections

Definition: JPA (Java Persistence API) and Hibernate help Java applications connect to databases and store, retrieve, update, and delete data.

Example: In a Student Management System, student details are stored in a MySQL database and retrieved using Java.

```
@Entity
public class Student {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
}
```

Explanation: @Entity maps the Student class to a database table, and @Id identifies the primary key.

2. Validation

Definition: Validation checks whether the input data is correct and follows specified rules before saving it to the database.

Example:

```
@NotBlank(message = "Name is required")

private String name;
```

Explanation: The student's name cannot be empty or contain only spaces.

3. Custom Error Handling

Definition: Custom error handling allows developers to display meaningful error messages when something goes wrong in an application.

Example:

```
@RestControllerAdvice

public class ErrorHandler {

    @ExceptionHandler(Exception.class)

    public String handleError(Exception e) {

        return "Something went wrong!";

    }

}
```

Output:

Something went wrong!

Explanation: When a matching exception occurs in a controller, the handler returns a simple error message. In a real application, specific exceptions and suitable HTTP status codes should be used.

Benefits

- JPA/Hibernate: Simplifies database operations.
- Validation: Prevents invalid input data.
- Custom Error Handling: Makes errors easier to understand and manage.

In simple words: JPA/Hibernate manages database data, validation checks input correctness, and custom error handling manages application errors.

Build a Spring Boot REST API for user management and CRUD operations, tested with Postman:

Spring Boot REST API for User Management and CRUD Operations

1. Simple Definition

A Spring Boot REST API for user management is an application that allows us to create, read, update, and delete user details using HTTP requests. Postman is used to test these API requests.

2. CRUD Operations

Operation	HTTP Method	Example
Create	POST	Add a new user
Read	GET	View user details
Update	PUT	Change user details
Delete	DELETE	Remove a user

3. Simple Java Example

@RestController

```
@RequestMapping("/users")
public class UserController {
    @GetMapping
    public String getUsers() {
        return "List of Users";
    }
    @PostMapping
    public String addUser() {
        return "User Added Successfully";
    }
    @PutMapping
    public String updateUser() {
        return "User Updated Successfully";
    }
    @DeleteMapping
    public String deleteUser() {
        return "User Deleted Successfully";
    }
}
```

Note: This is a simple demonstration. It returns messages and does not actually save, update, or delete database records.

4. Testing with Postman

POST /users

Adds a new user.

Create

GET /users

Displays users.

Read

PUT /users/1

Updates user 1.

Update

DELETE /users/1

Deletes user 1.

Delete

5. Example JSON Input

When testing the POST request in Postman, select Body → raw → JSON and enter:

```
{
  "id": 1,
  "name": "Srilatha",
  "email": "srilatha@example.com"
}
```

Expected response:

User Added Successfully

Benefits

- Simplifies user management.
- Supports all four CRUD operations.
- Makes API testing easy with Postman.
- Can be integrated with a MySQL database for permanent storage.

In simple words: We use Spring Boot to build APIs for managing user details and Postman to check whether the APIs work correctly.